Cheat Sheet

Every interview answer on one pass. Read this the morning of.

Day 1 — Core Java: The Language, Again

How Java Actually Runs (JDK / JRE / JVM)
"Java is compiled to bytecode by javac, and the JVM interprets or JIT-compiles that bytecode at runtime. That bytecode layer is what gives Java platform independence — the same class file runs anywhere a JVM exists."

Data Types, Variables & Wrappers
"There are 8 primitives with fixed sizes across platforms; everything else is a reference. Wrappers give me objects for collections and generics, and autoboxing converts automatically — but I always compare them with equals(), because == compares references and the Integer cache only covers -128 to 127."

Operators & Precedence
"Java operators follow C-like precedence. The two things I'm careful about: `++i` vs `i++` in expressions, and using `&&`/`||` deliberately for short-circuiting — `&` and `|` evaluate both operands and can turn a safe null check into an NPE."

Control Flow: switch, loops, break/continue
"I default to the modern arrow switch and switch expressions — no fall-through, it returns a value, and the compiler forces exhaustiveness. I know the classic colon form falls through without break, which is exactly why I avoid it."

Arrays
"Arrays are fixed-size, zero-indexed, and default-initialised. `arr.length` is a field, not a method. For anything dynamic I'd reach for ArrayList instead, but for coding rounds I'm comfortable with two-pointer and sliding-window patterns directly on arrays."

Strings, Immutability & the String Pool
"String is immutable — every modification returns a new object, which is why it's thread-safe and safe as a HashMap key. Literals are interned in the string pool and reused; `new String()` forces a separate object. I compare strings with equals(), never ==, and I use StringBuilder when I'm building strings in a loop."

Input / Output & Formatting
"I use Scanner for quick input but BufferedReader with StringTokenizer for large inputs, because Scanner is regex-based and slow. And I know the classic nextInt-then-nextLine newline bug — you consume the leftover newline or read lines and parse."

Stack, Heap & Pass-by-Value
"Java is strictly pass-by-value. When I pass an object, the value being copied is the reference — so mutating the object through that reference affects the original, but reassigning the parameter never changes the caller's variable. That's why a plain swap(x, y) method can't work."

Packages, Access Modifiers & the static Keyword
"static means the member belongs to the class rather than an instance — one shared copy, callable without an object, and it can't touch instance state or use this. Access modifiers go private → package-private → protected → public, widening at each step, and protected also grants access to subclasses in other packages."

Varargs, Enums & Command-Line Args
"Varargs is just an array parameter with a friendlier call site, and it must be last. Enums are full classes — constants are singletons, so == is safe, and they can carry fields, methods and even per-constant behaviour."

Day 2 — OOP, Collections, Exceptions & Generics

The 4 Pillars of OOP (with real examples)
"Encapsulation hides state behind behaviour, inheritance reuses code through an IS-A relationship, polymorphism lets the same call take many forms — overloading at compile time, overriding at runtime — and abstraction exposes what an object does via interfaces and abstract classes. In practice I favour composition over inheritance, because it keeps classes decoupled and testable."

Constructors, this & super
"Constructors have no return type and aren't inherited or overridable. this() and super() must be the first statement, so a constructor can only do one or the other. If I don't write one, the compiler inserts a call to super() — which is why a subclass fails to compile when the parent only has a parameterised constructor."

Overloading vs Overriding & Dynamic Dispatch
"Overloading is compile-time polymorphism — the compiler picks the method from the declared argument types. Overriding is runtime polymorphism — the JVM picks it from the actual object type. When overriding, the signature is identical, the return type can be a covariant subtype, I can widen access but never narrow it, and I can't throw broader checked exceptions. And I always add @Override so the compiler catches my typos."

Abstract Class vs Interface
"An abstract class models an IS-A relationship with shared state and a constructor — I use it for template-method style skeletons. An interface is a capability contract; since Java 8 it can carry default and static implementations, and a class can implement many of them, which is how Java supports multiple inheritance of type. For DI and testability I program to interfaces."

Object Methods & the equals/hashCode Contract
"Every class inherits from Object. The key contract: if two objects are equal their hash codes must be equal, so I always override equals and hashCode together, using the same fields. I use Objects.equals and Objects.hash for null safety, instanceof for the type check, and I keep those fields immutable so an object can't go missing inside a HashMap."

Exception Handling
"Throwable splits into Error, which I never catch, and Exception. Exceptions are checked — the compiler forces me to catch or declare them — or unchecked RuntimeExceptions, which usually indicate a bug I should fix rather than catch. I use try-with-resources so resources always close, I catch specific types from most- to least-specific, and when I wrap an exception I always pass the original as the cause."

Collections Framework: List & Set
"Collection splits into List, Set and Queue, and Map sits in its own hierarchy. ArrayList is my default — backed by a resizable array with O(1) random access; LinkedList only wins for frequent insertions at both ends. For unique elements I use HashSet for speed, LinkedHashSet when I need insertion order, and TreeSet when I need them sorted. HashSet relies on hashCode to find the bucket and equals to confirm the duplicate, which is why I override them together."

Map & HashMap Internals
"HashMap is an array of buckets indexed by hash(key) & (n-1). On a collision the entries are chained in a linked list, and since Java 8 a bucket with more than 8 entries is treeified into a red-black tree so the worst case stays O(log n). It resizes by doubling when size exceeds 0.75 × capacity. Keys must implement equals and hashCode properly and be immutable. For concurrency I'd use ConcurrentHashMap, not Hashtable — it locks per-bin and doesn't allow nulls."

Iterators, Comparable vs Comparator
"Comparable gives a class one natural ordering via compareTo; Comparator is an external strategy via compare, which is better when I need several orderings or can't modify the class. With Java 8 I compose comparators with comparing, thenComparing and reversed. For iteration, collection iterators are fail-fast — structural modification outside the iterator throws ConcurrentModificationException, so I use iterator.remove() or removeIf; concurrent collections are fail-safe instead."

Generics & Wildcards
"Generics give compile-time type safety and eliminate casts. Java implements them with type erasure — the type parameters are gone at runtime, so List<String> and List<Integer> are the same class, which is why I can't instantiate a type parameter or test instanceof on a parameterized type. For wildcards I follow PECS: use ? extends when I only read from the structure, ? super when I only write into it."

Day 3 — Modern Java: Streams, Concurrency, JDBC & Patterns

Lambdas & Method References
"A lambda is a concise instance of a functional interface — an interface with exactly one abstract method. Parameters types are inferred, and it can capture local variables as long as they're effectively final, because those values are copied. Method references are just shorthand when the lambda body only calls an existing method."

Streams API
"A stream is a lazy pipeline over a data source: a source, zero or more intermediate operations like filter and map, and one terminal operation that triggers execution and produces a result. It doesn't store data or modify the source, and it can only be consumed once. I reach for Collectors like groupingBy and joining to build results, and I only use parallelStream for large CPU-bound work because the overhead and ordering caveats usually aren't worth it."

Optional & the Date/Time API
"Optional is a container that may hold a value, designed as a return type so absence is handled explicitly — I use orElse, orElseGet, orElseThrow and map/flatMap chains instead of get(), and I never use it as a field or parameter. For dates I use java.time: it's immutable and thread-safe, unlike Date and Calendar, and DateTimeFormatter can be shared across threads where SimpleDateFormat can't."

Modern Java: 9 → 17 Features
"I'm comfortable on Java 11 and 17, the current LTS versions. Beyond Java 8 I use records for immutable DTOs instead of Lombok boilerplate, sealed interfaces with pattern-matching switch for closed hierarchies, var where the type is obvious, text blocks for SQL and JSON, and the Java 11 HTTP Client instead of HttpURLConnection."

Threads: Creation, Lifecycle & Basics
"A thread is a lightweight unit of execution inside a process; threads share heap memory but each has its own stack. I implement Runnable and call start() — never run() directly, which would just execute on the calling thread. For lifecycle: NEW, RUNNABLE, RUNNING, BLOCKED/WAITING/TIMED_WAITING, TERMINATED. And the key difference: sleep() keeps the lock, wait() releases it and must be called inside a synchronized block."

Synchronization, volatile & Deadlock
"synchronized provides both mutual exclusion and the visibility guarantees of happens-before, and it's reentrant. volatile gives visibility and prevents reordering but not atomicity, so I use it for flags and safe publication, never for compound operations like i++. For deadlock, the practical fix is consistent lock ordering, plus tryLock with a timeout; in production I'd diagnose with jstack or a profiler."

ExecutorService & Concurrent Collections
"I never create threads directly in production — I submit tasks to an ExecutorService so threads are reused and the pool can be bounded. I construct ThreadPoolExecutor explicitly rather than using the Executors factories, because the default unbounded queue can cause an OOM under load. For results I use Future or CompletableFuture when I need to compose async work, and ConcurrentHashMap or a BlockingQueue instead of synchronizing collections by hand."

File I/O & Serialization
"For file work I use NIO.2 — Files and Paths — which is cleaner and throws better exceptions than the old io streams, and I always close streams with try-with-resources. Serialization converts an object to bytes via the Serializable marker; I declare serialVersionUID to keep compatibility across versions and mark sensitive or non-serializable fields transient. In modern apps I'd usually serialize to JSON with Jackson instead."

JDBC — Connecting Java to a Database
"JDBC is the layer under JPA: get a connection from the pool, prepare a statement, bind parameters, execute, process the ResultSet and close everything in try-with-resources. I always use PreparedStatement — it's precompiled, faster on repeat, and critically it prevents SQL injection because the values are sent separately from the SQL. For writes I turn off auto-commit, commit on success and roll back on any exception."

Design Patterns You Must Know
"I know the classic GoF patterns by category — creational like Singleton, Factory and Builder; structural like Adapter, Decorator and Proxy; behavioural like Strategy, Observer and Template Method. For Singleton I'd normally use the enum implementation: the JVM guarantees a single instance and it's safe against reflection and serialization. And I can point at where Spring uses each — @Transactional is a proxy, JdbcTemplate is template method, events are observer."

Day 4 — Full Stack: Spring Boot, JPA, REST & SQL

Spring Core: IoC, DI & Beans
"IoC means the framework controls object creation and wiring instead of my code doing `new` everywhere. DI is how it's delivered — Spring injects dependencies, and I prefer constructor injection because it keeps fields final, makes dependencies explicit and makes unit testing trivial without Spring. A bean is any object managed by the container, singleton-scoped by default."

Spring Boot: Auto-configuration, Starters & Config
"Spring Boot is Spring with opinionated defaults. Its auto-configuration inspects the classpath and conditionally configures beans — Tomcat because spring-boot-starter-web is present, a DataSource because a JDBC driver is — but it backs off the moment I define my own bean. Starters bundle compatible dependency versions, the app runs as a plain JAR with an embedded server, and config is externalised into application.yml bound with @ConfigurationProperties and separated by profiles."

Spring MVC & REST Controllers
"A request hits the DispatcherServlet, which uses a HandlerMapping to find the controller method, binds parameters via HttpMessageConverters, runs the service layer and returns a value that Jackson serializes to JSON. I use constructor injection, return DTOs rather than entities so the API contract is stable and internal fields never leak, validate incoming bodies with @Valid, and centralise error responses in a @RestControllerAdvice so the client always gets a consistent JSON error shape."

JPA & Hibernate
"JPA is the specification and Hibernate is the implementation; Spring Data JPA generates repositories from interface method names. I map entities with @Entity/@Id/@GeneratedValue, set @ManyToOne to LAZY rather than accepting the EAGER default, and watch for the N+1 problem — which I fix with JOIN FETCH or @EntityGraph. Transactions belong on the service layer: Hibernate's dirty checking means I don't need to call save() on managed entities, and the commit happens when the transactional method exits."

SQL Essentials for Interviews
"I'm comfortable writing joins, group-by with having, subqueries and window functions. I know WHERE filters before grouping and HAVING after, that indexes speed reads but cost writes, and that a LEFT JOIN plus IS NULL finds unmatched rows. For slow queries I read the EXPLAIN plan first and look for full scans, functions on indexed columns and leading wildcards."

Maven, Build & Project Structure
"Maven uses a POM with GAV coordinates and a fixed lifecycle: validate, compile, test, package, verify, install, deploy. The Spring Boot parent POM manages dependency versions so I don't have to align them myself. I scope dependencies properly — test for JUnit/Mockito, provided for Lombok — and the spring-boot-maven-plugin produces an executable fat JAR I can run with java -jar."

Spring Security & JWT
"Spring Security is a filter chain in front of my controllers. For a stateless API I use JWTs: the user logs in, I verify the password with BCrypt, issue a signed token carrying subject, roles and expiry, and a filter validates that token on each request and populates the SecurityContext. I disable CSRF for stateless APIs but configure CORS explicitly, and I use @PreAuthorize for method-level authorization. Passwords are BCrypt-hashed with a per-user salt — never encrypted or stored in plain text."

Testing: JUnit 5 & Mockito
"I write JUnit 5 tests with Mockito: @Mock the dependencies and @InjectMocks the class under test, then given/when/then with AssertJ's fluent assertions. For Spring layers I use test slices — @WebMvcTest with MockMvc for controllers and @DataJpaTest for repositories — because they're far faster than loading the full context with @SpringBootTest. The pyramid I aim for is lots of unit tests, some integration tests, and a few end-to-end tests."

Microservices & Architecture Overview
"I'd start with a modular monolith and only split into microservices for a concrete reason — independent scaling, team autonomy or different availability needs. If we go distributed, each service owns its data, they communicate via REST or events, sit behind an API gateway, and need circuit breakers, centralised logging and distributed tracing, because partial failure and eventual consistency are the real costs."

Explaining Your Project (the round that decides it)
Structure beats talent here. Rehearse your 45-second pitch out loud today — problem, stack, your contribution, one hard problem with a measurable result, and one improvement. Then every bullet on your resume should survive two minutes of follow-up questions.

Rapid Fire — Rapid Fire: Traps, HR & Last-Minute Drill

Output Prediction Traps (Part 1: Strings & Numbers)
"String literals are interned in a pool and reused; `new String()` always creates a fresh object. Compile-time constants get folded, but concatenation involving a variable happens at runtime. Wrappers cache -128 to 127, so == works there and fails beyond it — which is why I always use equals()."

Output Prediction Traps (Part 2: OOP & Exceptions)
"Instance methods are dispatched at runtime by the object's type, but static methods are hidden and resolved at compile time by the reference type — and fields are never polymorphic at all. Initialisation runs static blocks once, parent before child, then instance blocks and constructors per object. And finally always runs; a return in finally overrides the try's return, though the returned value is snapshotted before finally executes."

One-Minute Answers (Verbal Rapid Fire)
"Answer in three beats: what it is, how it works, why it matters. Two sentences, then stop and let them ask for more."

HR & Behavioural Questions
"Every behavioural answer is a short story with a measurable result, and I never criticise a past employer or manager. My weaknesses are real but paired with what I'm doing about them, and I always have two questions about the team and its practices."

Final 24 Hours: What To Do & Skip
"Four days ago this all felt forgotten. You now have the language back, the framework story straight, and a project pitch that lands. Sleep tonight — recall beats cramming."